

# INTRODUCTION TO MACHINE LEARNING

MIHIR RAO

## Contents

|          |                                                                     |          |
|----------|---------------------------------------------------------------------|----------|
| <b>1</b> | <b>Introduction</b>                                                 | <b>2</b> |
| 1.1      | Classification: predicting a category . . . . .                     | 2        |
| 1.2      | Different classifiers fit different patterns . . . . .              | 3        |
| 1.3      | Digit images as points in a feature space . . . . .                 | 3        |
| 1.4      | Training error, test error, and overfitting . . . . .               | 4        |
| 1.5      | Validation: choosing a model without using the final test . . . . . | 5        |
| 1.6      | Supervised and unsupervised learning . . . . .                      | 6        |
| <b>2</b> | <b>Linear Classifiers and Perceptrons</b>                           | <b>7</b> |
| 2.1      | From observations to a decision function . . . . .                  | 7        |
| 2.2      | Level sets and linear boundaries . . . . .                          | 8        |
| 2.3      | Vectors, dot products, and hyperplanes . . . . .                    | 9        |
| 2.4      | Normals and signed distances . . . . .                              | 10       |
| 2.5      | The centroid method . . . . .                                       | 11       |
| 2.6      | Preparing the perceptron objective . . . . .                        | 12       |
| 2.7      | Risk and the optimization problem . . . . .                         | 13       |

---

## LECTURE 1: INTRODUCTION

---

MACHINE LEARNING uses patterns in data to make predictions. A model describes the patterns we are willing to consider; statistics helps us reason about them; an optimization algorithm fits the model to the data. The amount and quality of the data limit what can be learned. More data can also make methods effective that worked poorly with smaller collections.

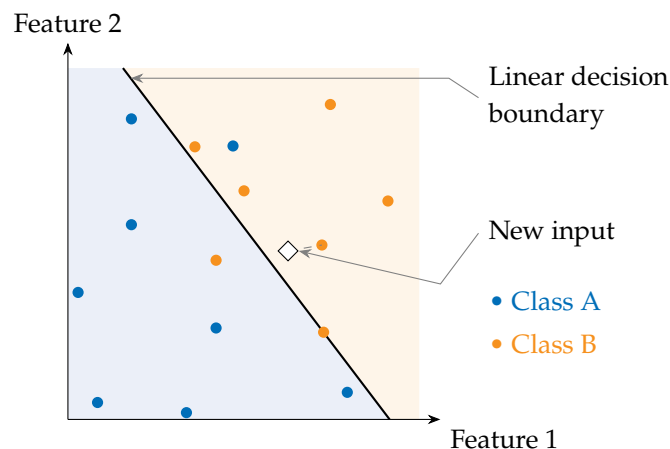
The central question is whether a learned pattern works on new examples. Remembering the examples already seen is not enough.

### 1.1 Classification: predicting a category

**Definition 1.1** [CLASSIFICATION]. CLASSIFICATION predicts which category, or CLASS, an input belongs to. A LABEL records the known class of an example. A CLASSIFIER is a rule that assigns a predicted class to an input.

**Example:** *Credit card default.* For each cardholder, record the balance and annual income, together with whether the cardholder defaulted. These labeled records are TRAINING DATA. Use them to learn a rule that predicts default for a new applicant. The two classes are “default” and “no default.” In this example, balances distinguish the groups more clearly than incomes do, although the groups still overlap.

The measured inputs are FEATURES. Collecting them in a FEATURE VECTOR makes each example a point: two features give a point in a plane. A DECISION BOUNDARY separates regions where the classifier predicts different classes. It separates *predictions*; it need not separate every observed label correctly.



**Figure 1:** A schematic classification problem. The colored points have known labels; the diamond is a new input. One rule uses the nearest labeled point, while another uses the side of a straight boundary.

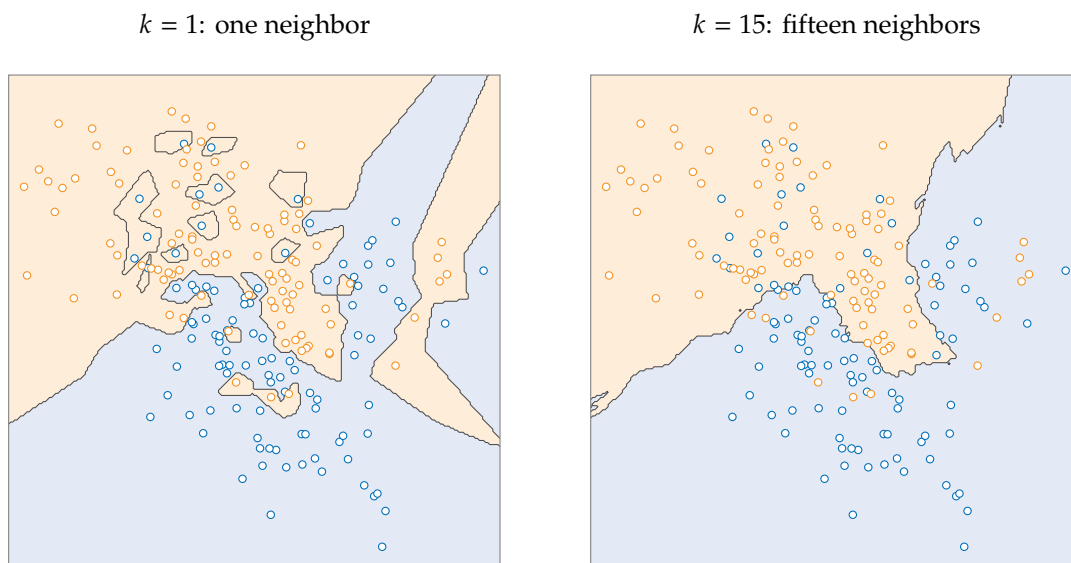
## 1.2 Different classifiers fit different patterns

The same training data can support different prediction rules.

**Definition:** The 1-NEAREST NEIGHBOR CLASSIFIER finds the training point closest to a new input and returns its label. The  $k$ -NEAREST NEIGHBOR CLASSIFIER instead finds the  $k$  closest training points and lets their labels vote. For two classes and odd  $k$ , one class has a majority.

At  $k = 1$ , the rule can respond to each individual training point. At  $k = 15$ , a point's label is balanced against fourteen other labels. This can give less intricate prediction regions, but may misclassify some training points. With distinct training inputs, 1-nearest neighbor classifies each training point correctly when that point itself is included among the possible neighbors: its distance to itself is zero. Conflicting labels at identical inputs would prevent this conclusion.

A LINEAR CLASSIFIER uses a straight decision boundary in two dimensions. This is a stronger restriction on the shape of the pattern. It can miss genuine nonlinear structure, but it also avoids the small islands that a nearest neighbor rule can create around individual points.



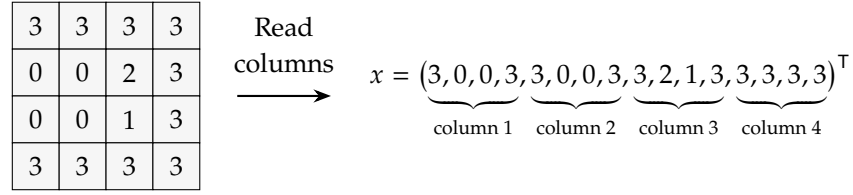
**Figure 2:** The same 200 labeled points classified with  $k = 1$  and  $k = 15$ . Background colors show predictions; circles show the training labels. At  $k = 1$ , small regions follow individual points. At  $k = 15$ , votes produce broader regions, and some circles lie in a region of the other color.

Correctly classifying all training points is therefore not enough to choose a rule. A very intricate boundary may capture accidental details rather than a pattern that will persist in new data.

## 1.3 Digit images as points in a feature space

**Example:** *Handwritten 7s and 1s.* Read the sixteen pixels of each  $4 \times 4$  image in a fixed order to obtain a point in  $\mathbb{R}^{16}$ . Labeled images of the two digits train a classifier.

4 × 4 pixel array



**Figure 3:** A 4 × 4 array becomes a sixteen-coordinate vector by reading down each column, from left to right. Training and new images must use the same coordinate order.

A **HYPERPLANE** is the higher-dimensional version of a straight line boundary. Thus a linear classifier for these images has a hyperplane as its decision boundary in sixteen-dimensional feature space.

The classification pipeline has two stages:

1. *Training*: compute feature vectors for the labeled images, then learn a classifier from the vectors and their labels.
2. *Prediction*: compute the feature vector of a new image in the same way, then evaluate the learned classifier on that vector.

## 1.4 Training error, test error, and overfitting

**Definition 1.2** [TRAINING AND TEST ERROR]. The **TRAINING ERROR** is the fraction of training examples that the classifier labels incorrectly. The **TEST ERROR** is the fraction labeled incorrectly in a separate collection of new examples that were not used to train the classifier.

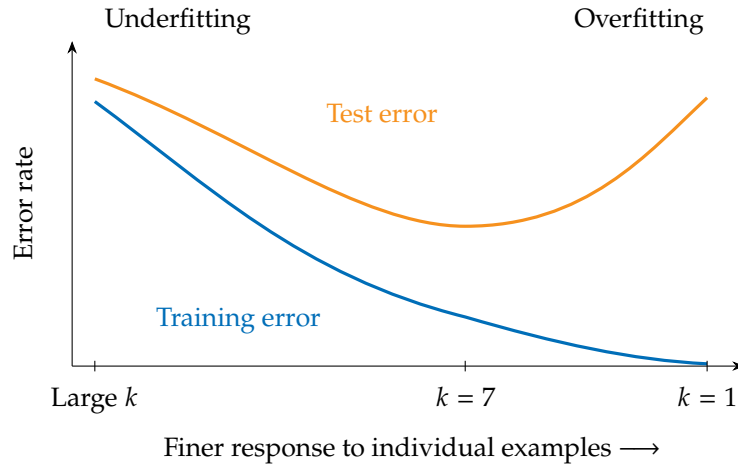
If a labeled set contains  $n$  examples, the error rate is

$$\text{error rate} = \frac{\text{number of incorrect predictions}}{n}.$$

Training error measures performance on familiar inputs. Test error checks **GENERALIZATION**: performance on new examples. The zero training error of 1-nearest neighbor says little by itself about generalization.

**Definition 1.3** [OUTLIERS AND OVERFITTING]. An **OUTLIER** is a point whose label is atypical for its location, such as a borrower who appears solvent but defaults anyway. **OVERFITTING** occurs when sensitivity to outliers or other accidental patterns makes performance on new data worse.

A decision boundary should capture useful patterns. Conversely, **UNDERFITTING** occurs when the rule is too restrictive to capture the useful pattern. Very large  $k$  can ignore local distinctions; very small  $k$  can react too strongly to individual examples.



**Figure 4:** Schematic error trends as  $k$  decreases and the rule responds to finer details. Training error can keep falling while error on new data rises. The illustrated comparison has its best test performance near  $k = 7$ ; this is an example, not a universal choice of  $k$ . No measured error rates are encoded by these curves.

## 1.5 Validation: choosing a model without using the final test

**Definition:** A **HYPERPARAMETER** is a setting that controls how a model is trained or how flexible its prediction rule is. The neighbor count  $k$  is one example. **VALIDATION** uses held-back labeled data to choose hyperparameters or compare models.

For a choice such as  $k$ , proceed as follows:

1. Hold back a validation set from the labeled data.
2. Train candidate classifiers with different settings, using the training set.
3. Compare their validation errors and choose the setting that performs best on the validation set.

The three sets have different jobs:

| Set        | Purpose                                                        |
|------------|----------------------------------------------------------------|
| Training   | Learn the model's fitted values, such as its weights.          |
| Validation | Choose settings such as $k$ , or choose among models.          |
| Test       | Evaluate the final choice once, after the choices are settled. |

Keep the test set out of tuning. If its results influence model selection, it no longer provides the intended final check. A plot of test error against  $k$  can explain overfitting; choosing  $k$  in a new experiment requires validation data.

Prediction competitions illustrate the same issue. Repeated choices can be guided by public feedback, while a private set supplies the final score. Much stronger public results than private results can indicate overfitting to the feedback used during development.

## 1.6 Supervised and unsupervised learning

SUPERVISED LEARNING uses examples with target outputs. In classification, the target is a category. In REGRESSION, it is a numerical value. UNSUPERVISED LEARNING seeks structure in the inputs without supplied target labels.

| Task                     | Example question                                      |
|--------------------------|-------------------------------------------------------|
| Classification           | Is this email spam?                                   |
| Regression               | How likely is this patient to have cancer?            |
| CLUSTERING               | Which DNA sequences are similar to each other?        |
| DIMENSIONALITY REDUCTION | What are common features and differences among faces? |

Clustering groups similar inputs. Dimensionality reduction represents important variation with fewer coordinates. Both concern structure in the inputs rather than predicting a supplied label.

**Revision.** Classification predicts a category from a feature vector. A decision boundary describes where the prediction changes. Nearest neighbor rules and linear rules impose different shapes on that boundary. Generalization is the goal: training accuracy alone cannot establish it.

*Common mistakes:* choosing a classifier only because its training error is zero; assuming smaller  $k$  always helps; changing the pixel order between training and prediction; or using the final test set to tune a model.

*Check yourself:* Why can 1-nearest neighbor have zero training error and still overfit? Why does a  $4 \times 4$  image have sixteen coordinates? Which set chooses  $k$ , and which set evaluates the final choice? What distinguishes supervised from unsupervised learning?

**Source.** Jonathan Shewchuk, University of California, Berkeley, CS 189/289A, *Introduction to Machine Learning*, Spring 2022, [Lecture 1: Introduction](#), PDF pp. 2–6. Explanatory steps are completed and figures redrawn. The neighbor comparison uses the 200 plotted points on PDF p. 4 (originally ESL, Figures 2.1–2.3); error curves are schematic. Administrative material and unrelated text within textbook crops are omitted.

---

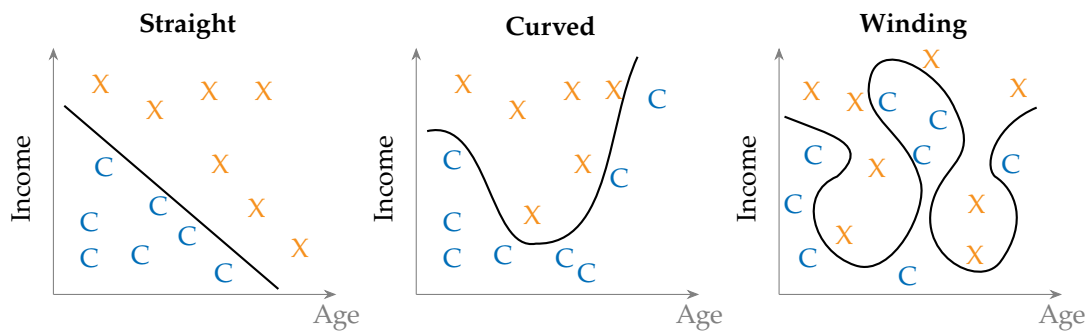
## LECTURE 2: LINEAR CLASSIFIERS AND PERCEPTRONS

---

### 2.1 From observations to a decision function

Suppose we have  $n$  observations, each described by  $d$  features, also called `PREDICTORS`. Some observations belong to a class  $C$ ; the others do not. Represent an observation by a feature vector  $x \in \mathbb{R}^d$ . Its coordinates are the input variables used for prediction.

For bank loans, the two features might be income and age. The class  $C$  contains borrowers who defaulted. A classifier uses these observations to predict whether a future borrower will default from their income and age. Different rules can produce very different boundaries.



**Figure 5:** Schematic loan data. A straight boundary suffices on the left. The middle arrangement needs a curved boundary. The winding boundary on the right fits the observed labels closely and risks overfitting.

A boundary partitions the input space into regions assigned different classes. A complicated boundary can fit the training observations well while making poor predictions on future observations.

**Definition:** A `DECISION FUNCTION`, or predictor function, maps an input  $x$  to a real score  $f(x)$ . Under the convention used here, the classifier predicts  $C$  when  $f(x) > 0$ , and predicts the other class when  $f(x) \leq 0$ .

For a continuous score that changes sign across its zeros, the decision boundary is the zero set

$$\{x \in \mathbb{R}^d : f(x) = 0\}.$$

This is typically a surface of dimension  $d - 1$ : a curve in two dimensions or a surface in three dimensions. Some classifiers specify their prediction regions without using a scalar decision function.

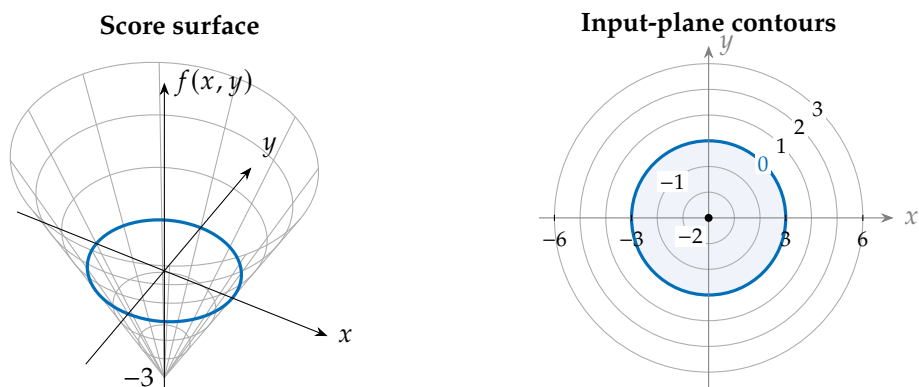
## 2.2 Level sets and linear boundaries

An **ISOSURFACE**, or level set, collects the inputs with the same score. For an **ISOVALUE**  $c$ , it is  $\{x : f(x) = c\}$ . The decision boundary uses  $c = 0$ ; other level sets help us see how the score changes away from it.

**Example:** Consider the radial score

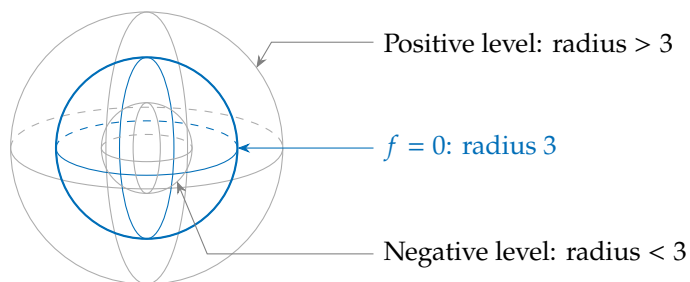
$$f(x, y) = \sqrt{x^2 + y^2} - 3.$$

The square root is the distance from the origin. Subtracting 3 makes the score zero on the circle of radius 3, negative inside that circle, and positive outside it. Its graph is a cone. Its level set for  $c \geq -3$  has radius  $c + 3$ ; at  $c = -3$  it reduces to the origin.



**Figure 6:** The same score viewed as a surface and as contours in the input plane. The blue circle is  $f = 0$ . The contours  $f = -2, -1, 1, 2, 3$  have radii 1, 2, 4, 5, 6, respectively.

In three dimensions, the analogous score  $f(x) = \|x\| - 3$  has concentric spherical level sets. The sphere of radius 3 is its decision boundary. The score is still a scalar, even though its input has three coordinates.



**Figure 7:** Spherical level sets in three-dimensional input space. Only the middle sphere, where  $f = 0$ , is the decision boundary.

A **LINEAR CLASSIFIER** has a flat decision boundary: a line in two dimensions, a plane in three dimensions, or a hyperplane in higher dimensions. We will describe it using  $f(x) = w \cdot x + \alpha$ .

## 2.3 Vectors, dot products, and hyperplanes

Write a vector as a column. For example, a point with five features is

$$x = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \end{bmatrix} = [x_1 \ x_2 \ x_3 \ x_4 \ x_5]^T.$$

The transpose  $T$  exchanges rows and columns. Lowercase Roman letters usually denote vectors; uppercase letters often denote matrices, sets, or random variables. Greek letters such as  $\alpha$  often denote scalars. Here  $n$  counts observations,  $d$  counts features, and  $i, j, k$  are indices. These are conventions, so context still matters.

The INNER PRODUCT, or dot product, multiplies corresponding coordinates and adds the results:

$$x \cdot y = \sum_{j=1}^d x_j y_j = x^T y.$$

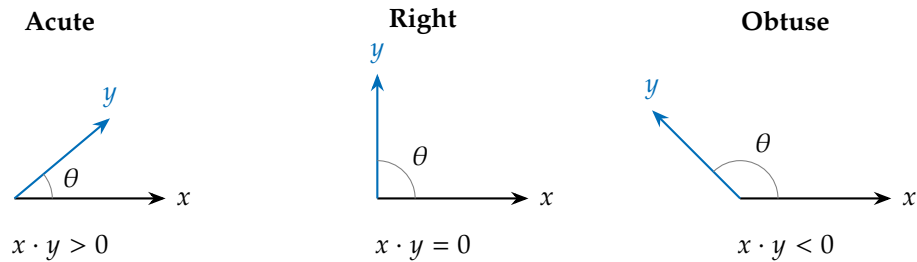
The EUCLIDEAN NORM measures a vector's length:

$$\|x\| = \sqrt{x \cdot x} = \sqrt{\sum_{j=1}^d x_j^2}.$$

To NORMALIZE a nonzero vector, divide it by its length. The result  $x/\|x\|$  is a UNIT VECTOR, with length 1. For nonzero  $x, y$  and the angle  $\theta$  between them,

$$\cos \theta = \frac{x \cdot y}{\|x\| \|y\|} = \frac{x}{\|x\|} \cdot \frac{y}{\|y\|}.$$

Thus the sign of the dot product tells us whether the angle is acute, right, or obtuse.



**Figure 8:** Dot products distinguish acute, perpendicular, and obtuse directions. The vectors must be nonzero for their angle to be defined.

**Definition 2.1** [HYPERPLANE]. For  $w \neq 0$ , the zero set of  $f(x) = w \cdot x + \alpha$  is

$$H = \{x \in \mathbb{R}^d : w \cdot x = -\alpha\}.$$

It is a flat, infinite set of dimension  $d - 1$  that divides  $\mathbb{R}^d$  into two half-spaces.

The score is commonly called linear. More precisely, it is **AFFINE** when the offset  $\alpha$  is nonzero; its boundary is still a hyperplane.

## 2.4 Normals and signed distances

The weights  $w$  determine which direction is perpendicular to the boundary. The offset  $\alpha$  determines its position.

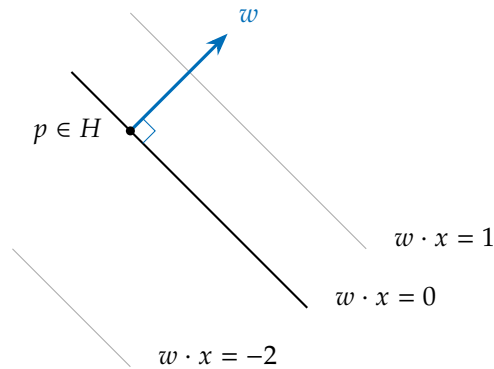
**Theorem 2.2** [NORMAL TO A HYPERPLANE]. If  $x, y \in H = \{u : w \cdot u = -\alpha\}$ , then  $w \cdot (y - x) = 0$ .

*Proof.* Subtract the two equations defining membership in  $H$ . Since  $w \cdot x = w \cdot y = -\alpha$ ,

$$w \cdot (y - x) = w \cdot y - w \cdot x = -\alpha - (-\alpha) = 0.$$

Every displacement between points on the hyperplane is therefore perpendicular to  $w$ . □

The vector  $w$  is a **NORMAL VECTOR** of  $H$ . Parallel hyperplanes can share the same normal while having different offsets.



**Figure 9:** Parallel lines with a common normal  $w$ . Moving in the direction of  $w$  increases  $w \cdot x$ ; moving along a line leaves it unchanged. The drawing is schematic.

The **SIGNED DISTANCE** is positive on the side toward which  $w$  points, and negative on the opposite side. For  $w \neq 0$ , it equals

$$s(x, H) = \frac{w \cdot x + \alpha}{\|w\|}.$$

To see this, choose a point  $p \in H$  and the unit normal  $u = w/\|w\|$ . The perpendicular component of  $x - p$  is

$$u \cdot (x - p) = \frac{w \cdot x - w \cdot p}{\|w\|} = \frac{w \cdot x + \alpha}{\|w\|}.$$

When  $\|w\| = 1$ , the score itself is the signed distance. Otherwise, divide both  $w$  and  $\alpha$  by  $\|w\|$  to obtain that interpretation without changing the predictions. The ordinary distance from the origin to  $H$  is  $|\alpha|/\|w\|$ . In particular,  $H$  passes through the origin exactly when  $\alpha = 0$ .

The entries of  $w$  and the offset  $\alpha$  are the classifier's **WEIGHTS**, also called parameters or regression coefficients. Data are **LINEARLY SEPARABLE** if a hyperplane puts the two classes on opposite sides. When no such hyperplane exists, a linear classifier cannot classify all training points correctly, though it may still predict most points well.

## 2.5 The centroid method

A simple rule summarizes each class by its average location. Let  $\mu_C$  be the mean of the training vectors in  $C$ , and  $\mu_X$  the mean of those outside  $C$ . These means are the two **CENTROIDS**; both classes must contain at least one point.

The **CENTROID METHOD** places the boundary halfway between these means, perpendicular to the segment joining them. Assuming  $\mu_C \neq \mu_X$ , set

$$w = \mu_C - \mu_X, \quad m = \frac{\mu_C + \mu_X}{2}, \quad \alpha = -w \cdot m.$$

Its score is

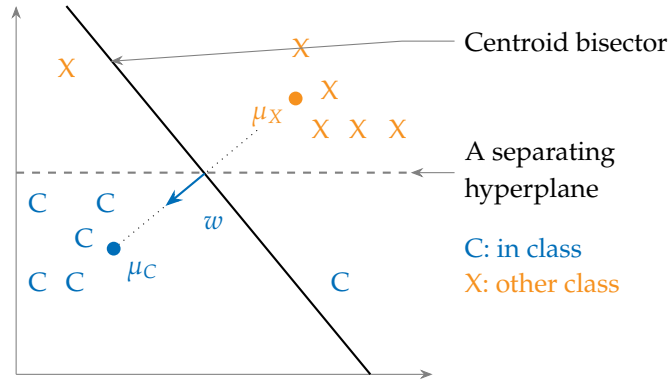
$$f(x) = (\mu_C - \mu_X) \cdot x - (\mu_C - \mu_X) \cdot \frac{\mu_C + \mu_X}{2} = w \cdot (x - m).$$

The midpoint  $m$  is on the boundary because  $f(m) = 0$ . The normal is the direction from  $\mu_X$  to  $\mu_C$ , so the positive side contains  $\mu_C$ .

This is also a nearest-centroid rule. Expanding the two squared distances and canceling  $x \cdot x$  gives

$$\begin{aligned} \|x - \mu_X\|^2 - \|x - \mu_C\|^2 &= 2(\mu_C - \mu_X) \cdot x + \|\mu_X\|^2 - \|\mu_C\|^2 \\ &= 2w \cdot (x - m) = 2f(x). \end{aligned}$$

Thus  $f(x) > 0$  means that  $x$  is closer to  $\mu_C$  than to  $\mu_X$ . Equal distances give score zero and use the other-class tie convention.



**Figure 10:** Centroid classification can fail on separable data. The centroid bisector misclassifies one point of each class, although the dashed horizontal line separates all points. The centroids and bisector are calculated from the schematic point positions.

Two means can hide important details of the data's arrangement. The rule can work well when the classes form two Gaussian clusters, but it need not find a separating hyperplane even when one exists. Changing only  $\alpha$  translates the boundary without rotating it; choosing a better offset can sometimes reduce training errors. If the centroids coincide, their difference is zero and this construction does not define a separating hyperplane.

## 2.6 Preparing the perceptron objective

THE PERCEPTRON, introduced by Frank Rosenblatt in 1957, uses numerical optimization to learn a linear classifier. Its training procedure can find a separator for linearly separable points, though it may be slow. The starting idea is to assign a penalty to violated classification constraints, then change the weights to reduce it.

Let the training vectors be  $X_1, \dots, X_n \in \mathbb{R}^d$ , with labels

$$y_i = \begin{cases} +1, & X_i \in C, \\ -1, & X_i \notin C. \end{cases}$$

The capital  $X_i$  identifies one observation. If observations are stored as rows of a data matrix  $X$ , its  $i$ th row is  $X_i^\top$ .

For now, restrict the boundary to pass through the origin:  $\alpha = 0$ . Then the score is  $z_i = X_i \cdot w$ . A positive label should have a positive score; a negative label should have a negative score. The relaxed, nonstrict constraints are

$$X_i \cdot w \geq 0 \quad (y_i = +1), \quad X_i \cdot w \leq 0 \quad (y_i = -1).$$

Multiplying by the label combines the two cases:

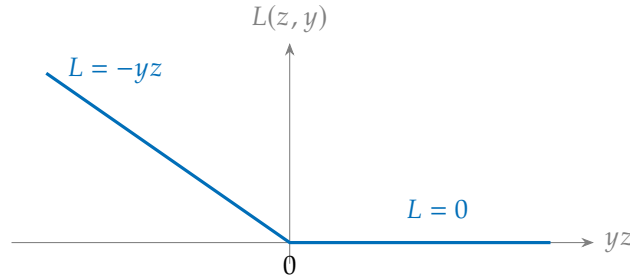
$$y_i X_i \cdot w \geq 0, \quad i = 1, \dots, n.$$

Each inequality is a **CONSTRAINT**. The product  $y_i z_i$  checks whether the score has the desired sign. Strictly positive products put every point on the correct side; a zero product puts a point on the boundary. Also,  $z_i$  is a signed distance only if  $\|w\| = 1$ .

**Definition:** The perceptron **LOSS FUNCTION** penalizes a negative label-score product:

$$L(z, y) = \begin{cases} 0, & yz \geq 0, \\ -yz, & yz < 0, \end{cases} \quad \text{equivalently} \quad L(z, y) = \max\{0, -yz\}.$$

Correctly signed scores have zero loss. A score with the wrong sign has positive loss, which increases as it moves farther in the wrong direction.



**Figure 11:** Loss as a function of the label-score product  $yz$ . Negative products are penalized linearly; zero and positive products have zero loss.

## 2.7 Risk and the optimization problem

Training combines the losses of all observations. The **RISK FUNCTION**, also called the objective or cost function, is their average:

$$R(w) = \frac{1}{n} \sum_{i=1}^n L(X_i \cdot w, y_i).$$

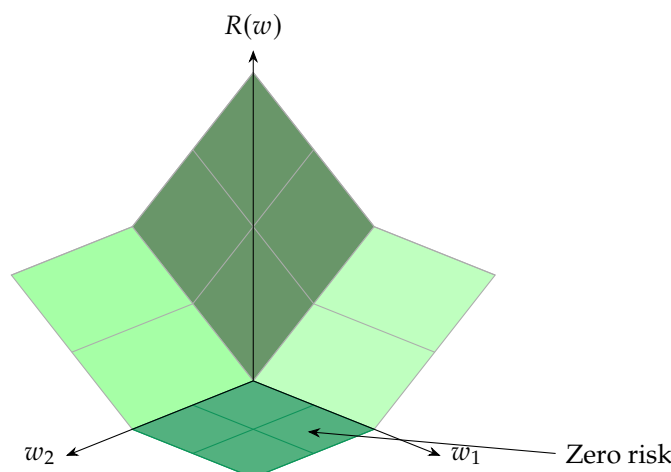
Let  $V = \{i : y_i X_i \cdot w < 0\}$  be the indices of violated constraints. All other terms have zero loss, so

$$R(w) = -\frac{1}{n} \sum_{i \in V} y_i X_i \cdot w.$$

Each term in this sum is positive. Hence  $R(w) > 0$  exactly when at least one of the nonstrict constraints is violated. The proposed optimization problem is

$$\underset{w \in \mathbb{R}^d}{\text{minimize}} \quad R(w).$$

The perceptron uses gradient-based steps to change  $w$  and reduce this objective. Here we have set up the constraints and the function to optimize.



**Figure 12:** Schematic risk surface for two weights. Flat green regions have  $R = 0$ ; sloping regions have positive loss. The shape depends on the training points and labels. No measured values are shown.

*Note.* Zero loss includes boundary ties. In particular,  $w = 0$  gives  $y_i X_i \cdot w = 0$  for every observation, so  $R(0) = 0$ , even though it does not give a useful separator. A zero score also predicts the other class under our convention, so a positive-label point on the boundary is not correctly classified. Therefore, zero risk certifies the *nonstrict inequalities*, not strict separation. Strictly positive products  $y_i X_i \cdot w$  certify that all points lie on the correct sides.

**Revision.** The zero set of  $w \cdot x + \alpha$  is a hyperplane with normal  $w \neq 0$ . Its signed distance is  $(w \cdot x + \alpha)/\|w\|$ . The centroid method uses the perpendicular bisector of the class means. The perceptron objective averages penalties for negative label-score products.

*Common mistakes:* treating a score as a distance without normalizing  $w$ ; forgetting the absolute value in the ordinary distance from the origin; assuming centroid classification succeeds on every separable data set; or treating zero perceptron risk as proof of strict separation.

*Check yourself:* Why is  $w$  perpendicular to the boundary? How does changing  $\alpha$  move it? Why does the centroid score compare distances to the means? Why does the loss vanish at  $w = 0$ ?

**Source.** Jonathan Shewchuk, University of California, Berkeley, CS 189/289A, *Introduction to Machine Learning*, Spring 2022, [Lecture 2: Linear Classifiers and Perceptrons](#), PDF pp. 1–6 (printed pp. 7–12). Figures are redrawn. The signed-distance derivation and centroid-distance identity complete the geometric arguments. The absolute value for ordinary distance and the zero-margin caveat make the stated formulas and classification convention consistent. Boundary and risk drawings are schematic.